

autonomous-QA — on-device :api-app backend bring-up & readiness notes

Scope: how scripts/autonomous-qa/lib-backend.sh::backend_up_apiapp should launch and detect readiness of the on-device Android API backend (:api-app, appId digital.vasic.lava.api / debug .dev) so the Phase 3 Android-backend matrix (qa_backend=apiapp) runs deterministically.

All facts below are read from source. Anything not provable from code is marked UNCONFIRMED:.

0. The one-line correction

The current lib-backend.sh TODO says the embed picks a **dynamic loopback port** and monkey -p ... LAUNCHER starts it. **Both are wrong:**

- The embed binds the **fixed default port 8443** (no override in production wiring — see §3). The "dynamic port" idea came from OnDeviceApiClient reading the port from the UI, which it does only for robustness; the produced value is always 8443.
 - A plain monkey ... LAUNCHER start launches MainActivity with a bare MAIN/LAUNCHER intent. MainActivity.handleIntent() only starts the engine when the intent carries the START_API extra (§1). So the current bring-up **launches the landing screen but never starts the server** — the sleep 25 then "succeeds" against a Stopped engine. This is the latent bluff the TODO warned about.
-

1. Exact adb commands to launch the server

Component: digital.vasic.lava.api.dev/lava.api.app.MainActivity (namespace = lava.api.app; debug applicationIdSuffix = .dev; MainActivity is android:exported="true" with the LAUNCHER filter — am start works from shell).

PKG=digital.vasic.lava.api.dev

1) Install the debug APK (already done by lib-backend.sh today).
adb -s "\$serial" install -r api-app/build/outputs/apk/debug/*.apk

2) Grant POST_NOTIFICATIONS BEFORE launch (API 33+). Without it, MainActivity.requestNotificationPermissionThenStart() pops the runtime permission dialog and only fires StartRequested in the dialog callback – so the engine would NOT start unattended. Harmless / errors on <API 33 (permission doesn't exist) → tolerate failure.
adb -s "\$serial" shell pm grant "\$PKG" android.permission.POST_NOTIFICATIONS || true

3) Launch MainActivity WITH the auto-start extra as a BOOLEAN (--ez).
MainActivity reads it via intent.getBooleanExtra(EXTRA_START_API, false), so the extra MUST be a boolean, not a string. This drives ApiControlAction.StartRequested → onStart() → controller.start(), which binds the TLS listener.
adb -s "\$serial" shell am start -W \
-n "\$PKG/lava.api.app.MainActivity" \
--ez lava.applink.START_API true

EXTRA_START_API literal value = lava.applink.START_API (AppLinkContract).

Cross-app note (UNCONFIRMED: as a bug, but relevant to the --ez choice): the client's OnboardingViewModel.onLaunchOnDeviceApi() does putExtra(EXTRA_START_API, "true") — a **String** — while MainActivity reads it with getBooleanExtra(...). A String extra read as a boolean returns the default (false). The adb path above sidesteps this by passing a real boolean (--ez). Whether the client→api-app auto-start actually fires in production is therefore UNCONFIRMED: and out of scope for this doc, but the orchestrator's --ez launch is correct regardless.

2. Deterministic readiness signal

There is **no dedicated readiness logcat marker and no readiness file** beyond the TLS cert. The Go embed logs nothing on successful Start; the Kotlin VM / controller log nothing on start (only Log.w on *failures*: ApiControlScreen, ApiApplication). So **do not poll logcat for "server up"**.

Recommended (real served-surface probe — the §6.B-correct signal): tunnel a host port to the device's loopback 8443 and poll GET /health, expecting HTTP 200. /health is registered **before** the Lava-Auth middleware in internal/router.Build (stated in mobile.go and proven empirically by Challenge C02, which gets 200 from /health with no key), so the probe needs no credential and no cert trust (curl -k).

```
hp="$(adb -s "$serial" forward tcp:0 tcp:8443)" # allocates a
        host port → device :8443
curl -ksf "https://127.0.0.1:${hp}/health" # 200 ⇔ embed
        bound + serving
adb -s "$serial" forward --remove "tcp:${hp}"
```

This is the exact analogue of backend_up_goapi's curl -ksf \$GOAPI_HEALTH loop, and it proves *both* that the listener bound the port *and* that the HTTP stack serves — not merely that the process launched.

Weaker proxy (fallback only): the embed persists its self-signed leaf at <filesDir>/lava-embed-cert.pem = /data/data/digital.vasic.lava.api.dev/files/lava-embed-cert.pem during Start, just before net.Listen. On the debuggable debug build: adb -s "\$serial" shell run-as "\$PKG" cat files/lava-embed-cert.pem non-empty ⇒ Start reached the TLS step. This is weaker than /health: it fires before net.Listen/Serve, so it cannot distinguish "bound + serving" from "about to bind". Prefer the /health probe.

Why adb forward works: the embed binds 0.0.0.0:8443 (wildcard, covers loopback); adb forward makes abdd open 127.0.0.1:8443 on the device, which the wildcard listener accepts. The forward is a **host-side probe convenience only** — the on-device client reaches the embed directly over loopback (§3) and needs no forward.

3. Loopback port + how the client selects the on-device API

Port = 8443, deterministic. Chain: ApiConfig.port default 8443 (core/apiengine/.../ApiEngine.kt) → ApiEngineController.DEFAULT_PORT = ApiConfig(sqlitePath="").port (8443) → ApiControlModule.provideController(...) passes **no** port arg → embed mobile.Start uses sc.Port (8443; sc.Port==0 would fall back to defaultPort=8443 anyway) → Status().port = 8443 → ApiControlState.Running.port = 8443.

Client onboarding "apiapp" path (so Challenge70 qa_backend=apiapp drives the right UI):

1. ApiSelection step → user taps the "On this device" button, whose `contentDescription` is **api-ondevice-launch** (`ApiSelectionStep.kt`). Challenge70's `selectApiBackend()` taps exactly this node for apiapp.
2. `OnboardingViewModel.onLaunchOnDeviceApi()` → `SiblingAppLauncher.intentToOpen()` + `START_API` + `RETURN_TO` extras → `OnboardingSideEffect.LaunchIntent` → screen `startActivity`s the api-app. **The button label/behaviour depends on the api-app being installed:** installed ⇒ "Open Lava API app" + a launch intent; not installed ⇒ "Install Lava API app" + a Firebase download intent (the WRONG path). So `backend_up_apiapp` **must install the api-app first** (it already does), and pre-starting it (§1) makes the key handoff immediately available.
3. The api-app starts the embed on `127.0.0.1:8443`. The client obtains `{access_key, loopback_port}` from the api-app's `ApiKeyProvider` `ContentProvider` (`content://digital.vasic.lava.api.dev.keyprovider`) via `ApiClient.read()` (and/or the `EXTRA_API_HOST=127.0.0.1` / `EXTRA_API_PORT` return extras). `loopback_port` = the Running port = 8443.
4. `OnboardingViewModel.onOnDeviceApiReturned(host, port)` builds `Endpoint.GoApi(host="127.0.0.1", port=8443, key=access_key)` and funnels into the normal `onSelectApi` → `probe` → `persist` → `advance` pipeline. (platform is only set as android when the endpoint is *discovered via mDNS*, not on the loopback handoff path.)

For the orchestrator: pre-starting the embed via §1 is **idempotent** with a later client-driven `START_API` re-launch — `ApiEngineController.start()` returns early when already Running/Starting (atomic compare-and-set guard), so no "address already in use" double-bind.

4. Cert / auth handling for the on-device API

OnDeviceApiClient (the api-app's OWN C02/C03 instrumentation client):

- Trusts **exactly** the leaf at `<filesDir>/lava-embed-cert.pem` via a custom `X509TrustManager` (no trust-all). It can read that file because the test runs **in the same process** as the embed (same `filesDir`).
- Connects to `https://127.0.0.1:<port>` — `127.0.0.1` is an **IP SAN** baked into the cert (`generateSelfSigned`), so default hostname verification stays ON.

- Auth: presents the access key in the **Lava-Auth** header (getWithKey(path, authKey, "Lava-Auth")). The embed's middleware base64-decodes + HMACs the value; /health needs no key, /index (and other /v1/...) return 401 without it.

Implication for Challenge70: Challenge70 drives the **client** app, not OnDeviceApiClient — it uses the client's own HTTP stack. The same-process cert trick does **not** apply cross-process: the client cannot read the api-app's private filesDir. Therefore whether Challenge70 can actually *complete a search* against the on-device embed depends on two things outside OnDeviceApiClient:

- the client being signed with the **same** keystore as the api-app (required for the signature-permission ApiKeyProvider read to succeed — both use the shared keystores/ block, so this holds for matched debug builds); and
 - **UNCONFIRMED:** how the client's network layer (under core/network) trusts the embed's self-signed leaf for the 127.0.0.1:8443 TLS handshake. Not read in this pass — verify before treating qa_backend=apiapp end-to-end search as gating. The /health readiness probe in §2 is unaffected (it uses curl -k).
-

5. Drop-in backend_up_apiapp (replaces the sleep 25 TODO)

Do NOT edit lib-backend.sh from this doc; this is the replacement body to apply later. It keeps the existing function contract (writes the apiapp LOCK on success, returns non-zero on failure). 8443 matches the literal already used for GOAPI_HEALTH in the same file (ApiConfig default port, §3).

```
backend_up_apiapp() {
    _backend_assert_free || return 1
    local serial="$1"
    local apiapp_port=8443 # ApiConfig.port default; production
                           wiring sets no override (§3)
    local apk
    apk="$(find "$REPO_ROOT/api-app/build/outputs/apk/debug" -name
        '*.apk' -type f 2>/dev/null | head -1 || true)"
    [[ -z "$apk" ]] && { echo "[backend] ERROR :api-app debug APK
        missing — run ./gradlew :api-app:assembleDebug" >&2;
        return 1; }

    echo "[backend] installing :api-app on $serial" >&2
    "$ADB" -s "$serial" install -r "$apk" >/dev/null

    # Grant POST_NOTIFICATIONS BEFORE launch so MainActivity fires
    StartRequested
```

```

# immediately instead of blocking on the runtime permission
# dialog (API 33+).
# Harmless / errors on <API 33 → tolerate.
"$ADB" -s "$serial" shell pm grant "$APIAPP_PKG"
    android.permission.POST_NOTIFICATIONS >/dev/null 2>&1 ||
    true

# Launch MainActivity WITH the auto-start extra as a BOOLEAN (--
# ez): MainActivity
# reads intent.getBooleanExtra(EXTRA_START_API). A plain
# LAUNCHER start does NOT
# start the embed – only START_API drives controller.start()
# (binds the listener).
echo "[backend] launching :api-app MainActivity with
    START_API=true" >&2
"$ADB" -s "$serial" shell am start -W \
    -n "$APIAPP_PKG/lava.api.app.MainActivity" \
    --ez lava.applink.START_API true >/dev/null

# Deterministic readiness: tunnel a host port to the device
# loopback:8443 and
# poll /health (pre-auth route) for 200 – the real served-
# surface signal
# ($6.B: process-launched != serving), mirroring
# backend_up_goapi.
local hp
hp="$("$ADB" -s "$serial" forward tcp:0 tcp:"$apiapp_port")" \
    || { echo "[backend] ERROR adb forward failed" >&2; return
        1; }
local waited=0 ok=0
while (( waited < 90 )); do
    if curl -ksf "https://127.0.0.1:${hp}/health" >/dev/null
        2>&1; then ok=1; break; fi
    sleep 3; waited=$((waited+3))
done
"$ADB" -s "$serial" forward --remove "tcp:${hp}" >/dev/null
    2>&1 || true

if (( ok == 0 )); then
    echo "[backend] ERROR apiapp /health never passed after $
        {waited}s. logcat tail:" >&2
    "$ADB" -s "$serial" logcat -d -t 200 2>/dev/null \
        | grep -iE 'lava|apiengine|AndroidRuntime' >&2 || true
    return 1
fi

echo "apiapp" > "$LOCK"
echo "[backend] apiapp healthy after ${waited}s (embed serving
    on device loopback :${apiapp_port}; client reaches it on-
    device)" >&2
}

```

backend_down_apiapp (existing: am force-stop + uninstall + rm LOCK)
needs no change; optionally add "\$ADB" -s "\$serial" forward --
remove-all for tidiness.